

SENTINELSHIELD

Advanced Intrusion Detection & Web Protection System

Technical Documentation, Practical Journal & Project Report

Prepared as a practical cybersecurity project

Version 1.0

1. Executive Summary

SentinelShield is an educational but realistic cybersecurity prototype that demonstrates the core workflow of a Web Application Firewall (WAF) and lightweight Intrusion Detection System (IDS). The system receives HTTP requests, collects relevant request information, applies rate-limiting controls, inspects request content against predefined attack signatures, makes an allow/block decision, records security events, generates alerts, and presents security information through a monitoring dashboard.

The project was designed around a practical security-operations workflow rather than only a collection of isolated scripts. This makes it suitable for demonstrating request inspection, signature-based detection, behavioral traffic controls, logging, alerting, dashboard interpretation, testing, and security analysis.

2. Project Purpose

The purpose of SentinelShield is to provide hands-on experience with the mechanisms used to protect web applications from common malicious requests and abusive traffic. It simulates the high-level behavior of a WAF in a controlled local environment.

3. Problem Statement

Web applications are exposed to HTTP requests that may contain malicious input, unexpected parameters, attack signatures, or excessive automated traffic. A security layer must inspect requests before they reach application functionality, identify suspicious behavior, apply a security decision, record the event, and provide visibility for analysis.

SentinelShield addresses this problem by implementing a simplified security pipeline that combines rule-based detection with IP-based rate limiting and security-event logging.

4. Objectives

Inspect incoming HTTP requests from a security perspective.

Identify common attack signatures using predefined rules.

Block requests classified as malicious.

Detect excessive request activity using IP-based rate limiting.

Generate structured security events and alerts.

Persist security events in a local log file.

Provide a dashboard for security-event monitoring.

Test the system using normal and controlled malicious requests.

Evaluate detection behavior using true-positive, true-negative, false-positive, and false-negative concepts.

Document limitations and propose future improvements.

5. Scope

The implemented prototype covers:

HTTP request collection and inspection.

Rule-based detection of common web attack categories.

Rate limiting based on source IP and a defined time window.

Blocking and HTTP response decisions.

Security event logging.

Alert generation.

Local security dashboard.

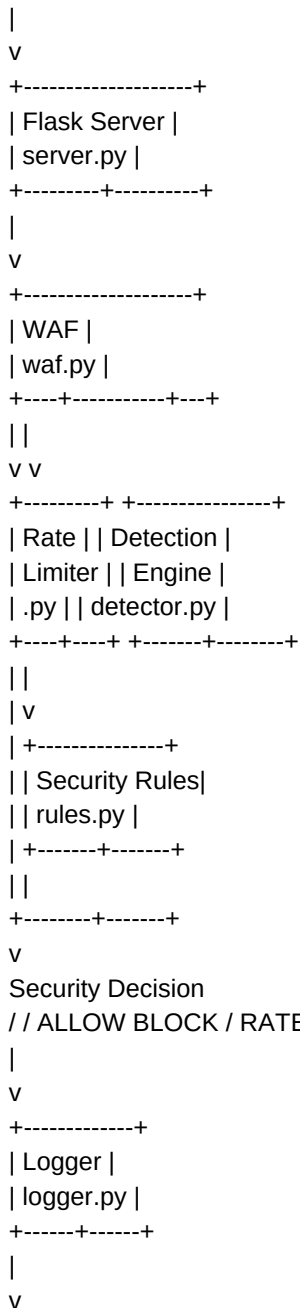
Controlled localhost testing.

The prototype does not claim to replace production WAF, IDS, SIEM, or endpoint-security products.

6. System Architecture

The SentinelShield architecture follows a layered security pipeline. Every request enters through the Flask server and is passed to the WAF. The WAF performs rate-limit and detection checks before returning an allow, block, or rate-limit decision.

Client / Test Tool



```
security.log
|
v
+-----+
| Dashboard |
| dashboard |
+-----+
```

7. Component Description

8. Security Processing Workflow

A client or test tool sends an HTTP request to the local Flask application.

The server collects the request method, URL/path, query parameters, form data, JSON data, headers, and source IP.

The WAF receives the collected request information.

The rate limiter checks whether the source IP has exceeded the configured request threshold.

If the threshold is exceeded, the request is rate-limited and an event is logged.

If the request passes the rate-limit check, the detection engine analyzes the request content.

The detection engine normalizes input and compares it with configured attack signatures.

If a malicious pattern is detected, the WAF blocks the request and generates a security event and alert.

If no malicious pattern is detected, the request is allowed.

Security events are written to the local security log and become visible on the dashboard.

9. Detection Capabilities

10. Request Normalization

The detection engine performs basic normalization before applying signatures. This includes conversion to lowercase, URL decoding, and whitespace normalization. Canonicalization improves the ability to recognize variations of known signatures, while the project acknowledges that simple signature matching cannot detect every possible obfuscation technique.

11. Rate Limiting and Behavioral Monitoring

SentinelShield implements an IP-based rate limiter. For the practical demonstration, the configured threshold is 10 requests within a 30-second window for a source IP.

This mechanism demonstrates how repeated traffic can be treated as potentially abusive even when individual requests do not contain a known malicious signature.

12. Logging and Alerting

Every blocked or rate-limited security event is recorded in logs/security.log. The log format contains the timestamp and structured key-value fields that can later be parsed by the dashboard.

```
2026-10-04 14:00:00 | IP=127.0.0.1 | Attack=Rate Limit Violation | Pattern=11 requests in 30 seconds |
Path=/protected | Action=RATE_LIMIT | Severity=MEDIUM
```

The logger also flushes file handlers after writing events so that events are immediately available for inspection during practical work.

13. Security Dashboard

The dashboard provides a centralized view of security activity. It reads the security log and summarizes total events, blocked attacks, rate-limit violations, attack categories, and recent events.

System status indicator.

Total security events.

Blocked attack count.

Rate-limit violation count.

Number of attack categories.

Attack-distribution visualization.

Recent security events with timestamp, IP, attack type, path, action, and severity.

[INSERT DASHBOARD SCREENSHOT HERE — Figure 1: SentinelShield Security Dashboard]

14. Installation and Execution

The project was developed and tested in a Windows local environment using a Python virtual environment.

14.1 Environment Setup

```
cd C:\Users\jagad\SentinelShield
.\venv\Scripts\python.exe -m pip install flask
```

14.2 Start the Application

```
.\venv\Scripts\python.exe app\server.py
```

The local application is available at:

<http://127.0.0.1:5000>

The dashboard is available at:

<http://127.0.0.1:5000/dashboard>

15. Practical Testing Procedure

Start the Flask server.

Open the protected endpoint and confirm that a harmless request returns HTTP 200.

Submit controlled SQL Injection, XSS, Path Traversal, LFI, and Command Injection test strings to the localhost application.

Confirm that malicious requests return HTTP 403.

Generate repeated local requests to demonstrate the rate limiter.

Confirm that requests beyond the configured threshold return HTTP 429.

Inspect logs/security.log for the generated security events.

Open the dashboard and verify that the events are summarized correctly.

Run the automated security test suite.

Run the final validation script and record the actual results.

16. Security Test Matrix

17. Final Validation

The `final_validation.py` script validates the core request-processing path against the local application. It checks normal traffic, malicious test cases, and the existence of the security log. The exact final pass count should be copied from the terminal output rather than estimated.

[INSERT FINAL VALIDATION TERMINAL SCREENSHOT HERE — Figure 2]

18. Detection Accuracy Analysis

The project uses the standard classification concepts below to evaluate the detection engine.

Accuracy is calculated as:

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN}) \times 100$$

The final report should use the actual values produced by `tests/tests_accuracy.py`. No accuracy figure should be invented or copied from an example run.

[INSERT ACTUAL ACCURACY TEST OUTPUT HERE — Figure 3]

19. Practical Evidence / Screenshot Register

20. Security Considerations

Testing is restricted to the local application and controlled laboratory traffic.

The prototype should not be deployed directly as a production security gateway without additional hardening.

A production system should use a mature WAF, secure deployment configuration, HTTPS, centralized logging, access controls, and monitoring.

Security rules should be maintained and tested to reduce false positives and false negatives.

Logs may contain sensitive request information in real deployments and therefore require appropriate access controls and retention policies.

21. Limitations

The detection engine is primarily signature-based.

Simple signatures can miss obfuscated or previously unseen attacks.

Basic normalization improves coverage but is not equivalent to full canonicalization.

Rate-limit state is maintained in memory and is therefore not suitable for distributed production deployment.

The log is stored locally rather than in a centralized SIEM or database.

The dashboard is a lightweight educational interface rather than a production SOC platform.

The project does not provide comprehensive protocol-level inspection or advanced anomaly detection.

22. Future Enhancements

Add a larger and version-controlled security-rule library.

Implement stronger canonicalization and tokenization.

Add anomaly-based and machine-learning detection.

Store events in a database for historical analysis.

Add automatic dashboard refresh and real-time event streaming.

Integrate with SIEM platforms.

Add authentication and role-based dashboard access.

Add email or messaging notifications for high-severity events.

Add IP reputation and threat-intelligence enrichment.

Add automated regression testing for every rule change.

Deploy behind HTTPS and a production WSGI server for controlled demonstration environments.

23. Expected Security Outcomes

A successful demonstration should show the following behavior: harmless traffic is allowed; recognized malicious patterns are blocked with HTTP 403; excessive traffic is rate-limited with HTTP 429; blocked and rate-limited events are written to the security log; and the dashboard reflects the resulting security activity.

24. Conclusion

SentinelShield successfully demonstrates the fundamental lifecycle of a lightweight web security gateway: request inspection, behavioral control, signature-based detection, security decision, logging, alerting, and monitoring. The project provides a practical representation of how security controls can be connected into an operational workflow rather than implemented as isolated features.

The project also demonstrates an important cybersecurity engineering principle: detection quality must be measured, not assumed. Testing with normal traffic and controlled attack patterns allows the system to be evaluated in terms of true positives, true negatives, false positives, and false negatives. Its documented limitations make clear that the prototype is intended for education and experimentation and would require additional controls before production use.

25. Viva / Interview Questions and Answers

Q: What is SentinelShield?

A: A lightweight educational WAF and intrusion-detection prototype that inspects HTTP requests, detects suspicious patterns, rate-limits abusive traffic, logs events, and presents them through a dashboard.

Q: What is a WAF?

A: A Web Application Firewall is a security control that inspects web traffic and can allow, block, or challenge requests based on security rules and behavior.

Q: Why is rate limiting used?

A: To restrict excessive request activity and reduce abuse such as brute-force attempts, flooding, or automated scanning.

Q: What is HTTP 403?

A: Forbidden. In this project it is used when SentinelShield blocks a request classified as malicious.

Q: What is HTTP 429?

A: Too Many Requests. In this project it indicates that a source exceeded the configured request threshold.

Q: What is a false positive?

A: A legitimate request incorrectly classified as malicious.

Q: What is a false negative?

A: A malicious request that the detector fails to identify.

Q: Why use logging?

A: Logging preserves security evidence for investigation, reporting, trend analysis, and dashboarding.

Q: Why use a dashboard?

A: It converts raw security events into summarized information that is easier for an analyst to monitor and interpret.

Q: Is SentinelShield production-ready?

A: No. It is an educational prototype. A production deployment would require stronger rule coverage, hardening, centralized logging, authentication, HTTPS, scalability, and additional security controls.

26. Appendix A — Project File Reference

27. Appendix B — Ethical Testing Statement

All attack-like inputs used in this project are controlled test strings submitted only to the student's local SentinelShield laboratory application. The tests are intended to demonstrate defensive detection and logging behavior and are not intended for use against systems without authorization.

28. Submission Checklist

Source code saved and organized.

Flask server successfully starts.

Normal request test completed.

Malicious request tests completed.

Rate-limit test completed.

Security log verified.

Dashboard verified.

Final validation executed.

Accuracy test executed and actual values recorded.

Screenshots inserted into the practical journal.

Final report reviewed for consistency.

No fabricated accuracy or detection results included.

Document prepared for SentinelShield project submission. The report describes the implemented educational WAF/IDS prototype, architecture, features, testing, deployment, and limitations.